Contents

- QUESTION 1 COMMENTING
- ==========
- 1. COMMENTING
- ==========
- ==========
- 2. SAMPLING IN IMAGES
- ==========
- QUESTION 4 VARIABLES (DO NOT CHANGE)
- ==========
- 4. AI for Sampling
- ==========
- ALL FUNCTIONS SUPPORTING THIS CODE %%

QUESTION 1 COMMENTING

DO NOT REMOVE THE LINE BELOW

```
clear; close all;
```

==========

1. COMMENTING

==========

MAKE SURE FILE BELOW IS IN SAME DIRECTORY AS THIS FILE

```
type('eel3135_lab03_comment.m')
```

```
% USER DEFINED VARIABLES
w = 20;           % Width
x = 0:1:79;       % Horiztonal Axis
y = 0:1:79;       % Vertical Axis

% defines an image matrix that transposes and uses the width and axes
z = round(exp(-1/w.^2*(((y.'-50)/1.5).^2+((x-20)).^2)));

% runs image systems with 3,6 and 90,-4 as scaling factors and shift
% factors
[xs,ys,zs] = image_system1(z,3,6);
za         = image_system2(zs,90,-4);

% PLOT RESULT WITH SUBPLOT
figure(1);
subplot(1,3,1);             % creates subplot 1
imagesc(x, y, z);           % display image matrix
axis square; axis image;        % scale and make proportional
title('Original')
subplot(1,3,2);             % creates subplot 2
imagesc(xs, ys, zs);        % display image matrix
axis square; axis image;        % scale and make proportional
```

```
title('After System 1')
subplot(1,3,3);             % creates subplot 3
imagesc(xs, ys, za);        % display image matrix
axis square; axis image;        % scale and make proportional
title('After System 2')


function [xs, ys, zs] = image_system1(z,Ux,Dy)
%IMAGE_SYSTEM1   scales an input matrix by Ux and Dy

% initialize matrix with Ux and Dy as dimensions
zs = zeros(ceil(size(z,2)/Dy),ceil(Ux*size(z,1)));

% scale original x and y vectors by Ux and Dy
ys = 1:ceil(size(z,1)/Dy);
xs = 1:ceil(Ux*size(z,2));

% forms new image
zs(1:end,1:Ux:end) = z(1:Dy:end,1:end);

end

function [za] = image_system2(z,Sx,Sy)
%IMAGE_SYSTEM2   shifts an image by Sx and Sy

% initializes za with same size as z
za = zeros(size(z,1), size(z,2));

for nn = 1:size(z,1)
        for mm = 1:size(z,2)
                % iterates through each element of z, if within bounds
                if nn > Sy && nn-Sy < size(z,1) && mm > Sx && mm-Sx < size(z,2)
                        % then shift by Sy and Sx
                        za(nn,mm) = 1/2*z(nn-Sy,mm-Sx);
                end
        end
end


end
```

==========

2. SAMPLING IN IMAGES

==========

```
% MAKE IMAGE
N = 64;     % image size
x = 0:N-1;  % x-axis
y = 0:N-1;  % y-axis
[xgrid,ygrid] = meshgrid(x,y);
b_min = 1; b_max = 32;
b = round( b_min + (b_max-b_min) * (xgrid/(N-1)) );   % spatially varying block size
z = mod( floor(xgrid./b) + floor(ygrid./b), 2 );      % final image


% ------------------------------------------------------------
% 2(a)
% ------------------------------------------------------------
% Only need to modify function -- this area can be empty
```

```
% ------------------------------------------------------------
% 2(b)
% ------------------------------------------------------------
[xs, ys, zs] = sample(z, 2);

figure;
subplot(1,2,1);
imagesc(x, y, z);
axis square; axis image;
title('Original')

subplot(1,2,2);
imagesc(xs, ys, zs);
axis square; axis image;
title('After sampling (D=2)')


% ------------------------------------------------------------
% 2(c)
% ------------------------------------------------------------
[xs, ys, zs] = sample(z, 4);

figure;
subplot(1,2,1);
imagesc(z);
axis image; axis square;
title('Original');

subplot(1,2,2);
imagesc(xs, ys, zs);
axis image; axis square;
title('After sampling (D = 4)');


% ------------------------------------------------------------
% 2(d)
% ------------------------------------------------------------
% ANSWER QUESTION BELOW
% The corners and the areas with the most noise and difference in color are
% the most affected because they are very close together and different
% which causes aliasing and loss of information, which makes the sampled
% image less accurate
%


% ------------------------------------------------------------
% 2(e)
% ------------------------------------------------------------
% Only need to modify function -- this area can be empty


% ------------------------------------------------------------
% 2(f)
% ------------------------------------------------------------
zaa = antialias(z);

figure;
subplot(1,2,1);
imagesc(x, y, z);
```

```
axis image; axis square;
title('Original');

subplot(1,2,2);
imagesc(x, y, zaa);
axis image; axis square;
title('Anti-aliased');


% ------------------------------------------------------------
% 2(g)
% ------------------------------------------------------------
[xs, ys, zs] = sample(z, 2);
[xs2, ys2, zaa_s] = sample(zaa, 2);

figure;
subplot(1,2,1);
imagesc(xs, ys, zs);
axis image; axis square;
title('Original sampled (D=2)');

subplot(1,2,2);
imagesc(xs2, ys2, zaa_s);
axis image; axis square;
title('Anti-aliased sampled (D=2)');


% ------------------------------------------------------------
% 2(h)
% ------------------------------------------------------------
[xs, ys, zs] = sample(z, 4);
[xs2, ys2, zaa_s] = sample(zaa, 4);

figure;
subplot(1,2,1);
imagesc(xs, ys, zs);
axis image; axis square;
title('Original sampled (D=4)');

subplot(1,2,2);
imagesc(xs2, ys2, zaa_s);
axis image; axis square;
title('Anti-aliased sampled (D=4)');


% ------------------------------------------------------------
% 2(i)
% ------------------------------------------------------------
% ANSWER QUESTION BELOW
% The anti-aliasing filter makes it so that if a part of the image is going
% to be aliased, and it is not sure whether it should be yellow or blue, it
% picks something in between to avoid being completely wrong about what
% should go in the cell. This makes the image more defined and accurate to
% the original.
%
```

QUESTION 4 VARIABLES (DO NOT CHANGE)

```
% LOAD VIDEO (MAKE SURE FILE BELOW IS IN THE DIRECTORY AS THIS FILE)
vid = VideoReader('wheel_video.mp4');
```

```
z = read(vid,[1 Inf]);

% The variable z is a 4 dimensional array, with dimensions 1 and 2 the
% m by n pixels in each color frame. The third dimension is the red,
% green, and blue colors in the image. The fourth dimension represents time

% HERE IS EXAMPLE CODE TO DISPLAY AND SAVE THE VIDEO AS AN MP4
% UNCOMMENT TO USE
%figure;
%for i = 1:min(size(z, 4),30*4)   % Only show 4 seconds max
%    tic; imagesc(uint8(z(:,:,:,i))); axis square; tm = toc;
%    pause(1/30-tm);              % Try to sync to 30 frames per second
%end
%v = VideoWriter('output_video','MPEG-4');
%open(v); writeVideo(v,uint8(z)); close(v)



% ------------------------------------------------------------------
% 3(a)
% ------------------------------------------------------------------
% Only need to modify function -- this area can be empty


% ------------------------------------------------------------------
% 3(b)
% ------------------------------------------------------------------

%30 hz / 1.875 = 16 = Dt
DX = size(z,2)/54;
DY = size(z,1)/90;

zs = video_sample(z, DX, DY, 16);

figure;
for i = 1:min(size(zs, 4),30*4)   % Only show 4 seconds max
    tic; imagesc(uint8(zs(:,:,:,i))); axis square; tm = toc;
    pause(1/30-tm);              % Try to sync to 30 frames per second
end
v = VideoWriter('wheel_stuck','MPEG-4');
open(v); writeVideo(v,uint8(zs)); close(v);


% ------------------------------------------------------------------
% 3(c)
% ------------------------------------------------------------------
% ANSWER QUESTION BELOW
% We want the sampling rate to match the apparent period (1 / (1.875Hz)) so
% that it looks like it doesnt repeat because it matches the period exactly.
% So then we divide by the actual period (1 / (30Hz)) to get the rate at
% which we should sample at to achieve the apparent period.
%


% ------------------------------------------------------------------
% 3(d)
% ------------------------------------------------------------------
zs_back = video_sample(z, DX, DY, 15);

figure;
for i = 1:min(size(zs_back, 4),30*4)   % Only show 4 seconds max
```

```
    tic; imagesc(uint8(zs_back(:,:,:,i))); axis square; tm = toc;
    pause(1/30-tm);              % Try to sync to 30 frames per second
end

v = VideoWriter('wheel_backwards','MPEG-4'); open(v);
writeVideo(v,uint8(zs_back)); close(v);


% ------------------------------------------------------------
% 3(e)
% ------------------------------------------------------------
% ANSWER QUESTION BELOW
% I know that undersampling causes distortion and moves the alias backwards
% so I sampled at a rate lower than 16 Hz. If 16 freezes it and stops it
% from moving, it makes sense that going over or under that value will
% cause it to move forwards or backwards.
```

==========

4. AI for Sampling

==========

---------- 4(a) ----------

```
%You are acting as a MATLAB signal-processing assistant.
%Modify the magic_image MATLAB function shown below to combine two grayscale
%images img1 and img2 of equal size into a single output image using two-dimensional sampling
%techniques.
%The goal of this function is to produce an output image that appears visually similar
%to img1 when viewed at full resolution, while revealing img2 when the image is subsampled.
%The signal must satisfy all of the following properties:
%1) The output image should appear visually similar to img1 when displayed normally
%at full resolution.
%2) When uniformly subsampled by selecting every 50th sample
%in both the horizontal and vertical directions, the resulting image should appear visually
%similar to img2.
%3) The spatial dimensions of the output image must be the same
%relative to the input images img1 and img2.
%The subsampling pattern must be uniform, go across every pixel, and applied equally in
%both spatial directions.
%*** Show your function here ***

function out = magic_image(img1, img2)
%MAGIC_IMAGE  Combine two identical-size gray-scale images.
%   out = MAGIC_IMAGE(img1,img2) returns an image that:
%     • Looks almost exactly like img1 at full resolution.
%     • Contains img2 when uniformly subsampled by selecting every 50th
%       pixel in both rows and columns.
%
%   Both inputs must be of type double and of the same size.
%
%   The method uses a simple 2-D high-frequency carrier that
%   retains the carrier's average at zero – thus it is invisible
%   when viewed normally – but after down- it collapses
%   to the low-frequency version of the carrier, revealing img2.

    % Ensure the inputs are double and same size
    assert(isequal(size(img1), size(img2)), ...
        'img1 and img2 must be the same size.');
```

```
    sz = size(img1);

    % ------- Step 1: Construct a spatial carrier ----------
    % We use a checkerboard of period 50 (any even period is fine).
    % The carrier has zero mean and alternates between +1 and -1.
    p = 50;                                          % subsampling factor
    [X,Y] = ndgrid(0:sz(1)-1, 0:sz(2)-1);            % index grids
    % Checkerboard pattern: period p, value +1/-1
    carrier = (-1).^( floor(X/p) + floor(Y/p) );

    % Normalise the carrier to zero mean (+1/-1 pattern already has avg 0)
    % You can scale down the strength of the carrier if you wish
    carrierStrength = 0.5;                          % controls the visibility at full res
    carrier = carrierStrength * carrier;

    % ------- Step 2: Embed img2 in the carrier -------------
    % img2 is modulated by the carrier and added to img1.
    % At full resolution the carrier averages to 0,
    % so the appearance is dominated by img1.
    out = img1 + carrier .* (img2 - 0.5*(max(img2(:))+min(img2(:))));

    % ------- Step 3: Optional clipping to valid range ------
    out = max(min(out, 1), 0);    % assume images are in [0,1]
end


% ---------------------------------------------------------------
% 4(b)
% ---------------------------------------------------------------
img1 = im2double(imread('img1.jpg'));
img2 = im2double(imread('img2.jpg'));


magic_img = magic_image(img1, img2);

[xs, ys, zs] = sample(magic_img, 5);

figure;
subplot(1,2,1);
imagesc(magic_img);
axis image; axis off;
title('Magic image original');

subplot(1,2,2);
imagesc(xs, ys, zs);
axis image; axis off;
title('Magic image sampled (D=5)');
```

ALL FUNCTIONS SUPPORTING THIS CODE %%

```
function [xs, ys, zs] = sample(z, D)
%SAMPLE   sample by keeping every Dth pixel

zs = z(1:D:end, 1:D:end);

ys = 1:size(zs,1);
xs = 1:size(zs,2);

end
```

```
function zaa = antialias(z)
%ANTIALIAS   make antialiasing filter for image z

zaa = zeros(size(z,1), size(z,2));

    for nn = 1:size(z,1)
        for mm = 1:size(z,2)
            x_minus = max(nn-1, 1); %if nn-1 < 1, matlab will throw error so use 1 if it is <1
            x_plus  = min(nn+1, size(z,1)); %same for if it is greater than size (stack smashing?)
            y_minus = max(mm-1, 1);
            y_plus  = min(mm+1, size(z,2));
            %mm = y, nn = x

            zaa(nn, mm) = (1/5) * ( z(x_minus, mm) + z(nn, y_minus) + z(nn, mm) + z(nn, y_plus) + z(x_plus, mm) );
        end
    end

end

function zs = video_sample(z, Dx, Dy, Dt)
%VIDEO_SAMPLE   samples video with axes and time

zs = z(1:Dy:end, 1:Dx:end, :, 1:Dt:end); %sample by x pixels until the end

end
```

Published with MATLAB® R2024a